Congestión — Guía 5

RCCD 2C 2026 · Resumen de la teórica 15 (Congestión), la práctica de control de congestión en TCP y las definiciones de la guía 5.

0 · La idea en criollo (ELI5)

Pensá la red como una autopista con peajes (los routers). Cada peaje tiene una playa de espera (el **buffer**). Si llegan más autos de los que el peaje despacha, la playa se llena y los autos que siguen llegando **se tiran** (drop tail). Los conductores a los que les tiraron el auto vuelven a mandarlo (retransmisión), y eso agrega todavía más tráfico. Eso es la **congestión**: un problema que se realimenta solo y, en el límite, llega al **colapso**, donde la red está a full y casi nada útil llega a destino (goodput → 0).

Nadie le avisa a TCP "che, la red está llena". Entonces TCP **tantea**:

- Arranca mandando poquito y, mientras le vuelvan los ACKs, **duplica** lo que manda en cada RTT (**Slow Start**, que de lento no tiene nada).
- Cuando se acerca a un umbral (**SSTHRESH**, "hasta acá era seguro la última vez") pasa a crecer **de a un segmento por RTT** (**Congestion Avoidance**).
- Si **se pierde algo**, asume que fue por congestión: baja el umbral a la mitad de lo que tenía en vuelo y frena. Si se enteró por **timeout** (grave: no vuelve nada), arranca de cero. Si se enteró por **3 ACKs duplicados** (leve: los datos siguen llegando), solo corta a la mitad (**FR/FR**).

Todo eso se resume en una variable, la **ventana de congestión CWND**: cuántos bytes me animo a tener "en el aire" sin que me los hayan confirmado. Y nunca puedo pasarme de lo que el receptor dice que puede recibir (**RWND**, el control de flujo).

> **Control de flujo vs. control de congestión.** El de flujo protege al **receptor** (punto a punto, RWND/AdvertisedWindow). El de congestión protege a la **red** (efecto global, CWND). TCP manda `min(CWND, RWND)`.

1 · Teórica: congestión en general

Definición y síntomas

Congestión: estado de sobrecarga *sostenida* donde la demanda de recursos (enlaces y buffers) está al límite de la capacidad o la supera. Se nota como QoS degradada. Los síntomas son pérdida de paquetes (los buffers se llenan) y retardos que crecen (encolamiento). Las métricas que la delatan son el % de descartes, el largo medio de cola, la cantidad de retransmisiones/timeouts, y el delay promedio y su desvío.

Causas: mucho tráfico hacia una misma salida, cuellos de botella "aguas abajo" (varias líneas rápidas que desembocan en una lenta), procesadores lentos. **Más memoria no lo resuelve**: con buffers infinitos los paquetes esperan tanto que el emisor da timeout y retransmite duplicados, que también ocupan la red. Se cambia pérdida por delay y se empeora.

Los nodos son **FIFO + Drop Tail** salvo que se diga otra cosa. Combinados pueden causar **sincronización global**: muchas conexiones pierden a la vez, todas bajan a la vez y todas vuelven a subir a la vez.

Carga vs. performance

- Al subir la carga, el throughput sube lineal hasta el **knee**. Desde ahí el tiempo de respuesta crece rápido. En el **cliff** el throughput se desploma (colapso).
- **Potencia = throughput / delay**. Su máximo marca la carga óptima.
- **Equidad de Jain**: $f = (\Sigma x_i)^2 / (n \cdot \Sigma x_i^2)$. Vale 1 si todos reciben lo mismo y 1/n si uno solo se lleva todo.

Colas M/M/1 (un buffer)

Llegadas Poisson con tasa λ, servicio exponencial con tasa μ, un servidor, cola infinita. La **intensidad** es ρ = λ/μ. Solo hay estado estacionario si **ρ < 1**; si ρ ≥ 1 el sistema desborda (congestión).

`E[N] = ρ/(1−ρ)  E[T] = E[N]/λ = 1/(μ−λ)  (Little: N = λ·T)`

E[T] explota cuando ρ → 1: por eso no conviene operar cerca del 100% de utilización. Con M/M/m (más servidores) la curva aguanta más, pero igual explota.

Soluciones y taxonomía (Yang & Reddy)

- **Pre-asignar recursos** (reservar buffers/BW): evita la congestión pero **subutiliza**. Con datagramas (IP) no es viable porque no hay circuito establecido sobre el cual reservar. Eso es lo que pregunta el ej. 1a.
- **Lazo abierto**: se decide de antemano, sin mirar el estado de la red (conmutación de circuitos, GMPLS).
- **Lazo cerrado**: se usa realimentación (redes de paquetes). Hay dos tipos:
  - **Implícita** (end-to-end): la fuente *infiere* la congestión a partir de timeouts o ACKs duplicados. Es la que usa **TCP** (Tahoe, Reno, Vegas, CUBIC). Es simple y vive en el nivel de transporte.
  - **Explícita** (network-assisted): el router *marca* los paquetes. Ejemplos: DECbit, **ECN** (bits en el byte ToS de IP), ATM ABR, ICMP source quench. Es más precisa, pero hay que cambiar tanto los hosts como los routers.

RED — Random Early Detection (AQM)

Es una técnica del **router** que *previene* la congestión: en vez de esperar a que el buffer se llene, descarta (o marca con ECN) paquetes al azar **antes**, para que las fuentes TCP frenen de a una y no todas juntas (evita la sincronización global). No guarda estado por conexión.

- Promedio de la cola: `AvgLen = (1−w)·AvgLen + w·SampleLen`, con 0 < w < 1.
- AvgLen ≤ MinTh → encola. MinTh < AvgLen < MaxTh → descarta con probabilidad P. AvgLen ≥ MaxTh → descarta siempre.
- `TempP = MaxP·(AvgLen−MinTh)/(MaxTh−MinTh)`, `P = TempP/(1 − count·TempP)`. Valores típicos: MaxP ≈ 0.02 (uno de cada 50) y MaxTh = 2·MinTh.
- La probabilidad de que le descarten a un flujo es ≈ proporcional a la porción del ancho de banda que usa.

Throughput de TCP en régimen (Mathis)

En CA la ventana oscila entre W/2 y W (diente de sierra). Por ciclo se mandan $3W^2/8$ paquetes en W/2 RTTs. Con probabilidad de pérdida p (1 pérdida cada 1/p paquetes) queda $W = \sqrt{(8/(3p))}$ y

`BW = (MSS / RTT) · C / √p  con C = √(3/2)`

Moraleja: el throughput es **inversamente proporcional al RTT y a √p**.

2 · Control de congestión en TCP (RFC 5681, convenciones de la guía)

Variables

Nombre	Qué es	Valor inicial (guía)
SMSS	tamaño máximo de segmento del emisor (sin headers)	2 KB
CWND	ventana de congestión: cuánto me animo a tener en vuelo	IW = 2·SMSS = 4 KB
SSTHRESH	umbral entre Slow Start y Congestion Avoidance	64 KB
RWND	lo que anuncia el receptor (AdvertisedWindow)	dato del ejercicio
FlightSize	bytes enviados y todavía no ACKeados (LastByteSent − LastByteAcked)	—

```
MaxWindow = min(RWND, CWND) EffectiveWindow = MaxWindow - (LastByteSent - LastByteAcked)
```

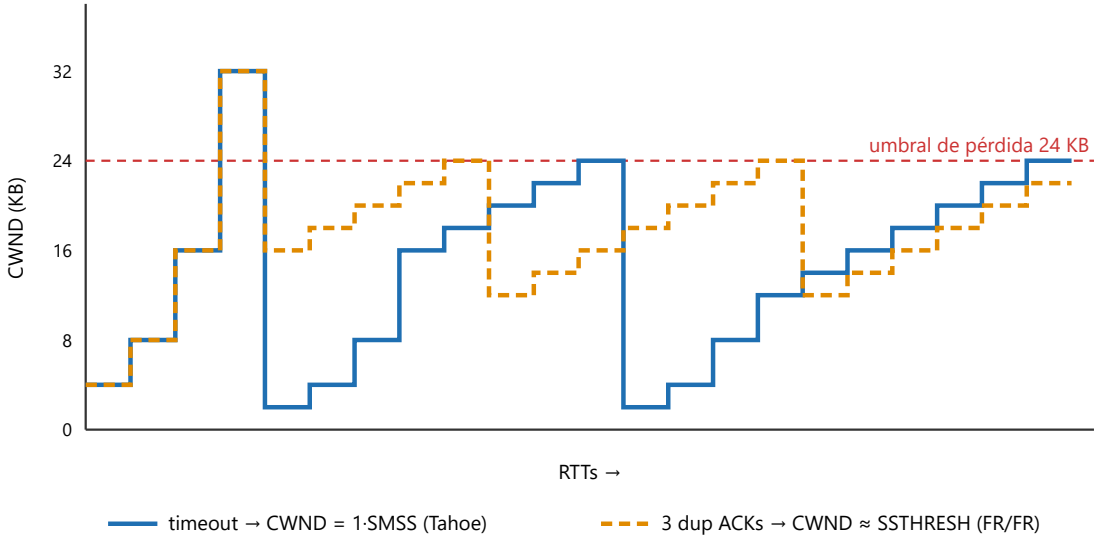
Reglas — la tabla que hay que saber de memoria

Situación	SSTHRESH	CWND
Slow Start (CWND < SSTHRESH)	igual	+min(N, SMSS) por ACK ⇒ **×2 por RTT**
Congestion Avoidance (CWND > SSTHRESH)	igual	+SMSS·SMSS/CWND por ACK ⇒ **+1 SMSS por RTT**
Timeout	max(FlightSize/2, 2·SMSS)	**1 SMSS** (LW), y vuelve a Slow Start
3 ACKs duplicados (entra a FR/FR)	max(FlightSize/2, 2·SMSS)	SSTHRESH + 3·SMSS; retransmite el segmento perdido
Cada ACK dup. extra en FR/FR	igual	+1 SMSS (infla la ventana: cada dup = un segmento que salió de la red)
ACK de datos nuevos (sale de FR/FR)	igual	**= SSTHRESH** y sigue en CA
Idle ≥ 1 RTO sin transmitir	igual	min(IW, CWND)

Trampas típicas.

- El SSTHRESH nuevo sale de **FlightSize/2**, no de CWND/2. Si la app mandaba menos de lo que permitía la ventana (o RWND limitaba), FlightSize < CWND. El ej. 12 se basa justamente en esto.
- Lo que sale a la red por RTT es **min(CWND, RWND)**. CWND puede seguir creciendo en papel aunque RWND sea la que limita.
- En Slow Start la duplicación **no se pasa de SSTHRESH**: al alcanzarlo, recorto ahí y sigo en CA. Es la convención habitual; si no está clara, aclarala en la respuesta.
- Si se pierde **toda la ráfaga**, no vuelve ningún ACK, ni siquiera duplicado. Eso es **timeout**, no FR/FR.
- Los ACKs son **acumulativos y en bytes**: el ACK dice "el próximo byte que espero".
- Con ACKs demorados (1 ACK cada 2 segmentos), en Slow Start CWND crece menos que ×2 por RTT, porque suma de a ACK (eso es min(N, SMSS)). Lo pregunta el ej. 7.

Cómo se ve (ejemplo inventado: SSTHRESH 64 KB, ráfagas ≥ 24 KB se pierden enteras)



Traza en azul (timeouts), en KB por RTT: `4, 8, 16, 32✗` → SSTHRESH = 32/2 = 16, CWND = 2 → `2, 4, 8, 16, 18, 20, 22, 24✗` → SSTHRESH = 12 → `2, 4, 8, 12 (recortado), 14, …, 24✗` → SSTHRESH = 12 otra vez. SSTHRESH ya no cambia: eso es el **estado estacionario** del ej. 11.

La curva naranja es lo que pasaría si la pérdida se detectara por 3 ACKs duplicados (FR/FR). No vuelve a 1 SMSS, sino a la mitad: es el diente de sierra AIMD "puro". Es lo que te pide comparar el ej. 8.

RTT y RTO

Versión de la guía (la original, la que se usa en los ejercicios):

`RTT[i+1] = α·RTT[i] + (1−α)·SampleRTT RTO = 2·RTT estimado`

- Con α = 1 el estimado nunca se mueve: el RTO queda clavado en 2·(primera muestra).
- Con α chico el estimado sigue a la última muestra (reacciona rápido pero es ruidoso).
- Hay **timeout** si una muestra (un ACK) tarda más que el RTO vigente *en ese momento*.

Karn/Partridge: no se samplea el RTT de un segmento retransmitido (sería ambiguo a qué envío corresponde el ACK) y, al retransmitir, se **duplica el RTO** (backoff).

Jacobson/Karels (RFC 6298) tiene en cuenta la varianza:

`Diff = Sample − Est; Est += δ·Diff; Dev += δ·(|Diff| − Dev); RTO = μ·Est + ϕ·Dev (μ=1, ϕ=4, δ=1/8)`

3 · Tipos de ejercicio de la guía y cómo se encaran

A Conceptuales — ej. 1, 2, 3, 7

Son preguntas de "¿se puede…? / ¿por qué…?". La receta: decir **qué mecanismo** está en juego y **qué pasa en el borde**.

- **Pre-alocar buffers con datagramas** → no hay conexión sobre la cual reservar, y si reservás para todos, subutilizás.
- **Ruteo solo** → el ruteo puede repartir la carga, pero si la demanda total supera la capacidad, no hay ruta que lo salve.
- **Buffers infinitos** → no hay pérdidas, pero el delay crece sin cota, salta el timeout y hay retransmisiones duplicadas: más carga y peor.
- **Cuándo CWND no representa la congestión**: pérdidas por **errores de bit** (enlaces inalámbricos), donde TCP frena sin que haya congestión; conexiones muy cortas o **idle**, donde CWND quedó vieja; o cuando lo que limita es RWND o la app. Para los primeros casos, las medidas son ECN/feedback explícito o retransmisión en el nivel de enlace.
- **¿Por qué no arrancar con una ventana enorme?** Porque no conocés la capacidad del camino: una ráfaga grande llena los buffers de golpe y genera pérdidas para vos y para todos. Era el problema original del colapso de 1986 que motivó a Jacobson.
- **CA**: +1 SMSS por RTT.

B ACKs acumulativos con segmentos fuera de orden — ej. 4

Método: numerá los bytes (1–100, 101–250, 251–450, 451–750). El receptor manda un ACK por cada segmento que llega, con el valor **"próximo byte que espero"**. Si llega un segmento después de un hueco, se repite el ACK anterior (ACK duplicado). Cuando llega el que faltaba, el ACK salta hasta el final de lo contiguo. Después aplicá Slow Start a cada ACK: `CWND += min(N, SMSS)`, donde N son los bytes **nuevos** que reconoce ese ACK (los duplicados reconocen 0 bytes nuevos). Revisá también si llegan a ser 3 duplicados (acá no: hay menos segmentos).

C Estimación de RTT / RTO — ej. 5

Armá una tabla con estas columnas: `i | Sample | RTT est. | RTO`. Arrancá con RTT_0 = primera muestra (lo dice el enunciado). Para cada ACK nuevo, primero chequeá si **Sample > RTO vigente** (si es así, hubo timeout) y recién después actualizá el estimado y el RTO. Repetí para cada α. Lo que el ejercicio quiere que veas: con α alto el RTO es estable pero lento para reaccionar; con α bajo sigue a la última muestra, así que después de una muestra chica (20 ms) el RTO baja mucho y la siguiente muestra grande da timeout.

D Trazas de CWND por RTT — ej. 6, 9, 10, 11 (los más "de parcial")

Siempre la misma tabla:

RTT	CWND	SSTHRESH	se envía = min(CWND, RWND, lo que falta)	¿pérdida?	acumulado
1	4	64	4	no	4
…	×2 o +2				

- Cada fila es **1 RTT**, y el tiempo total es (cantidad de filas) × RTT. Por ejemplo, en el ej. 6 contás cuántos RTT hacen falta hasta que CWND ≥ RWND (24 KB): 4 → 8 → 16 → 32, recortado por RWND = 24.
- "**Ráfagas de X KB o más se pierden enteras**" → si lo que se envía en ese RTT es ≥ X, esa fila es una pérdida por **timeout**: SSTHRESH = max(Flight/2, 4 KB), CWND = 2 KB, y esos datos **no cuentan** como enviados (se reenvían).
- Si RWND < X, **nunca** llegás a congestionar: CWND queda topeada por RWND. Es la clave del ej. 11b: anunciar *menos* ventana (18 KB < 20 KB) puede hacer que la transferencia sea **más rápida**.
- **Estado estacionario**: el momento en que SSTHRESH deja de cambiar entre un ciclo y el siguiente.

- El tiempo que se pierde esperando el timeout depende de qué asumas para el RTO. Lo normal es contarlo como un RTT más (o lo que diga el enunciado). **Escribí el supuesto.**
- Ej. 9 (1 GB): hacé un par de ciclos hasta el estacionario, calculá **bytes por ciclo** y **RTTs por ciclo**, y multiplicá. Sin congestión: llegás a RWND = 64 KB y después mandás 64 KB por RTT.

E Graficar — ej. 8

Graficá CWND en función del tiempo, con una pérdida cada 1 s, en dos versiones. **Sin FR/FR** cada pérdida es timeout: caída a 1 SMSS, rampa exponencial hasta SSTHRESH y después lineal (curva azul). **Con FR/FR**, caída a SSTHRESH = Flight/2 y lineal (curva naranja, diente de sierra). En ambos casos marcá el valor de SSTHRESH después de cada pérdida.

F Interpretar una tabla — ej. 12

Lo que hay que mirar: **CWND vs. FlightSize** (si FlightSize < CWND, la ventana no es lo que limita: la app no tiene datos o RWND es menor), **los saltos de CWND** (×2 es SS, +1 SMSS es CA, caída a 1 SMSS es timeout, caída a la mitad es FR/FR) y **LBS** (LastByteSent: cuánto avanzó realmente el envío). Con la regla FlightSize/2 calculás qué SSTHRESH quedó después de la caída, y con eso explicás las filas siguientes.

> **Para escribir en el parcial:** antes de los números, poné siempre los supuestos (ACK por segmento, convención en el borde CWND = SSTHRESH, cuánto dura el timeout, si contás en KB o en segmentos). En esta materia la justificación vale tanto como el número.